



STRING CALCULATOR

TEST-DRIVEN DEVELOPMENT

INDICATIONS AVANT DE COMMENCER

Ne pas lire les éléments en avance.

Faire une tâche à la fois, l'important est d'apprendre à travailler incrémentalement.

Faire les tests seulement pour les valeurs d'entrée valides.

Il n'est pas nécessaire de prévoir les tests pour les valeurs invalides.

Enfin, appliquer le TDD tout au long du TP.

ATELIER

1. Créer le squelette de la fonction `add(string numbers) : int`

a. La fonction peut prendre 0, 1 ou 2 nombres, et retournera la somme.

Pour une chaîne de caractères vide, la fonction `add` retournera 0.

Voici quelques exemples :

```
"" == 0 , "1" == 1, "1,2" == 3
```

b. Commencer par le test unitaire le plus simple, le cas où la chaîne de caractères est vide, puis les cas de un et deux nombres.

c. Résoudre les cas le plus simplement possible, cela vous permettra d'écrire des tests auxquels vous n'avez pas pensé.

d. Ne pas oublier de refactoriser le code après chaque tests passés.

2. TDD : Améliorer la fonction `add` pour qu'elle puisse gérer une infinité de nombres.

3. TDD : Améliorer la méthode `add` pour qu'elle comprenne le retour à la ligne (à la place de la virgule)

a. L'entrée suivante est correcte : `"1\n2,3" == 6`

b. L'entrée suivante est incorrecte : `"1,\n"` (ce n'est pas la peine d'écrire un test pour ce cas)

4. TDD : Supporter différents séparateurs (ang. delimiters)

Pour définir un séparateur, la chaîne de caractères contiendra le modèle ci-suit :

```
"//[delimiter]\n[numbers...]"
```

Exemple : `"//;\n1;2" == 3`

Pour des raisons de simplicité, les crochets ne sont pas des séparateurs valides.

5. TDD : Appeler la fonction `add` avec un nombre négatif lèvera l'exception suivante :

```
"Negatives not allowed. [negative number]"
```

6. TDD : S'il y a plusieurs nombre négatifs, les afficher tous dans le message d'exception.

7. TDD : Les nombres strictement plus grand que 1000 doivent être ignorés.

Exemple : `"1\n2,1002" == 3`

8. TDD : Les séparateurs peuvent être définis par plusieurs caractères avec le format suivant :

`"//[delimiter]\n[numbers...]"`

Exemple : `"//[***]\n1***2***3" == 6`

9. TDD : Il est possible de définir plusieurs séparateurs avec le format suivant :

`"//[delimiter1][delimiter2]\n[numbers...]"`

Exemple : `"//[*][%]\n1*2%3" == 6`

10. TDD : Enfin, vérifier que le cas suivant fonctionne.

`"//[**][%%]\n1**2%%3" == 6`